

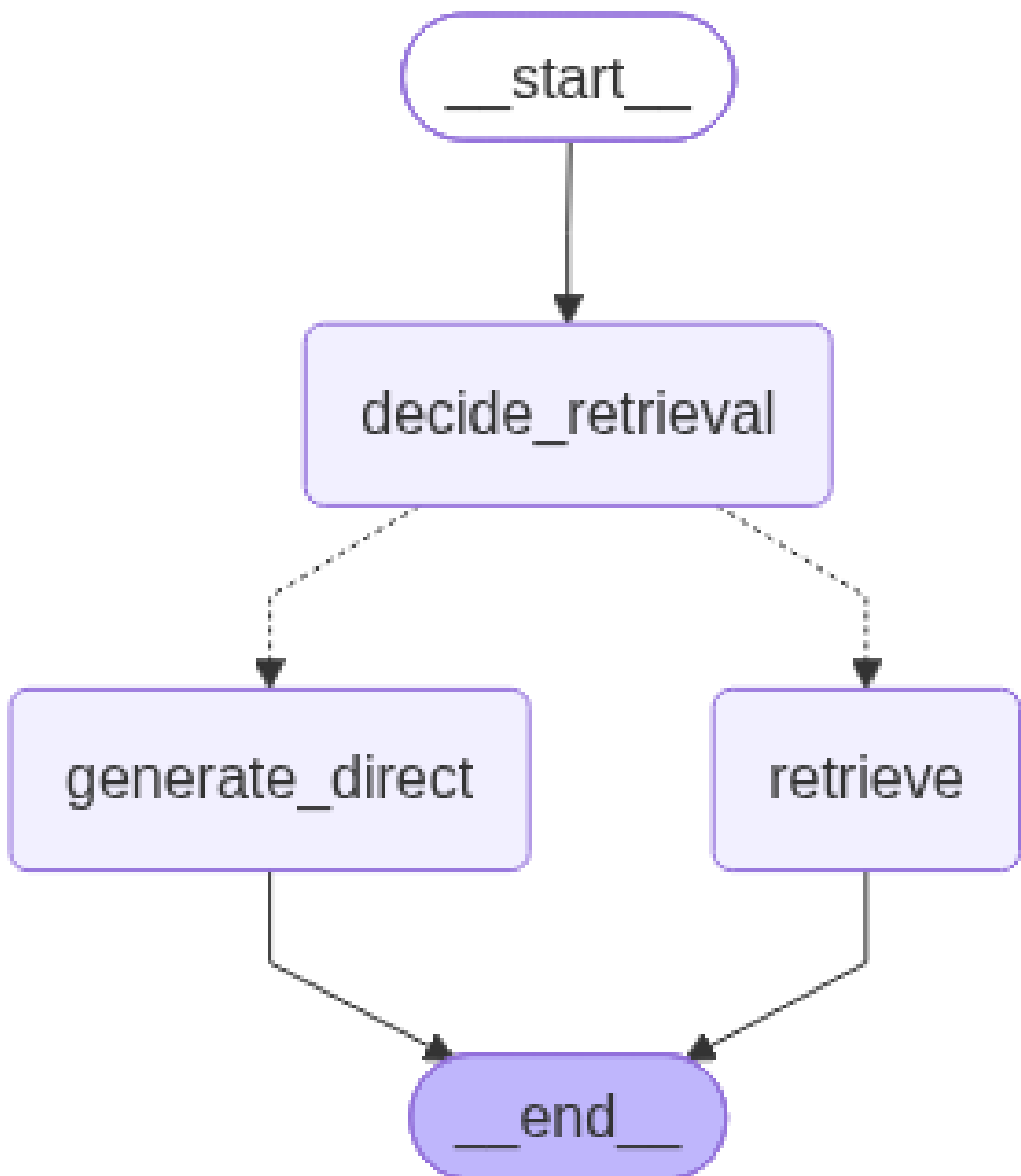
Self-RAG Architecture

A Step-by-Step Build, Step 1 through Step 7
(with Step 3 web-search variants 3.0 / 3.1 / 3.2)

Each page shows what was added at that step and the resulting LangGraph flow diagram.

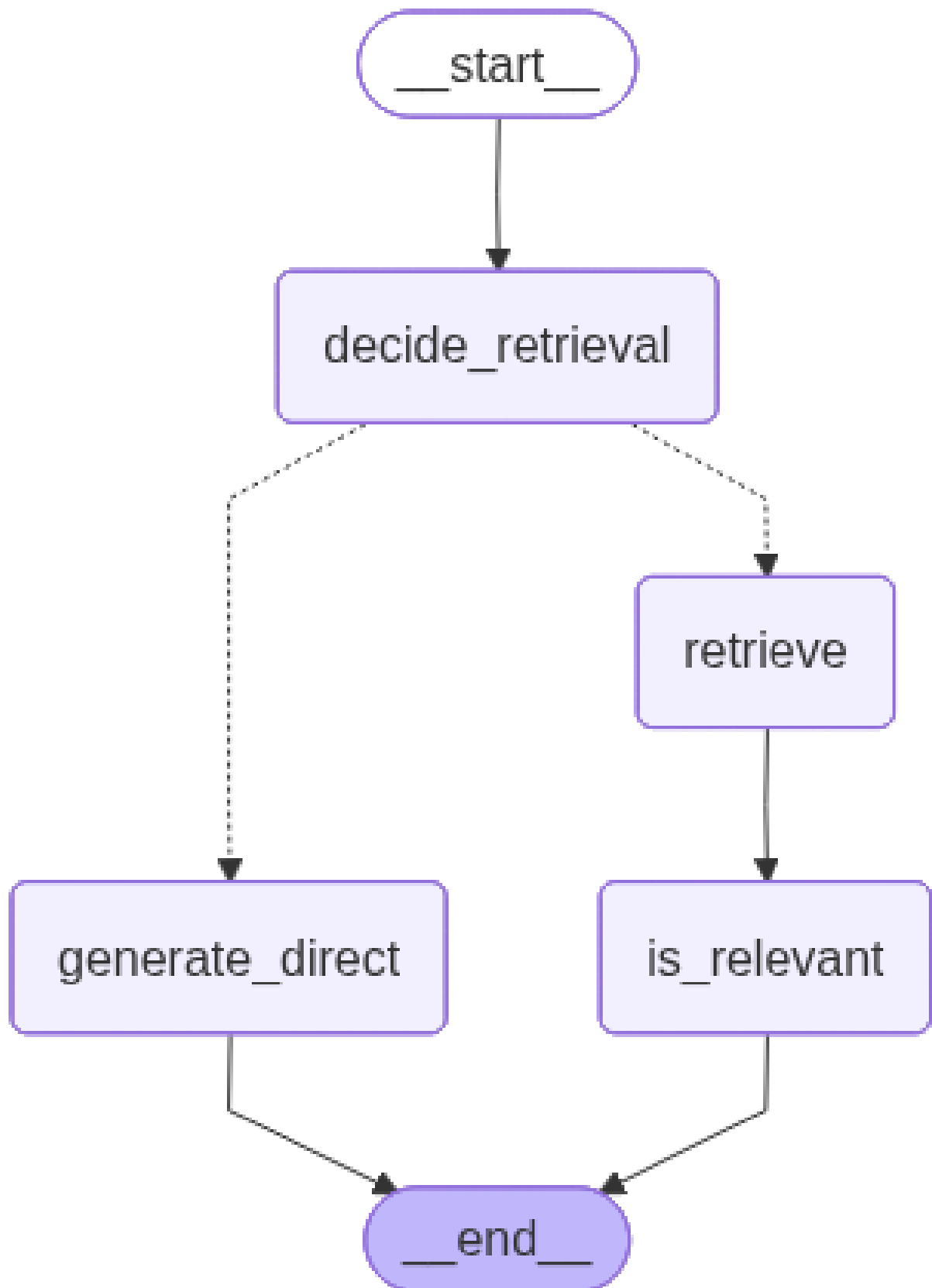
Step 1 - Adaptive Retrieval Skeleton

- `decide_retrieval`: an LLM decides whether the question needs company documents or can be answered from general knowledge.
- If no retrieval is needed -> `generate_direct` answers straight away using only the model's own knowledge.
- If retrieval is needed -> `retrieve` pulls chunks from the FAISS vector store, but the path just ends here for now (no filtering or answer generation yet). This is the routing scaffold everything else builds on.



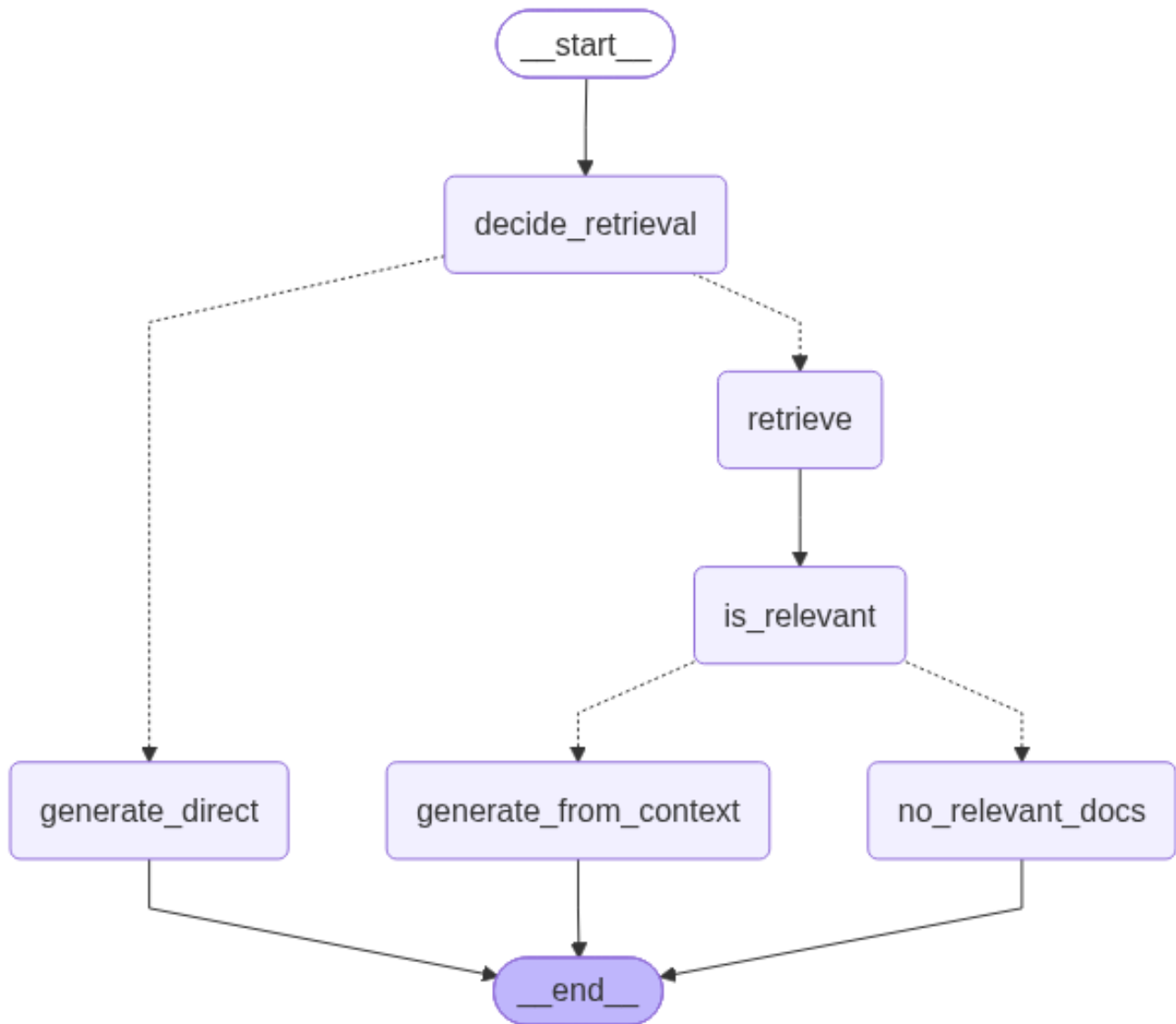
Step 2 - Document Relevance Filtering

- Adds is_relevant: after retrieval, an LLM judges each retrieved chunk individually and keeps only the ones useful for the question in relevant_docs.
- The retrieval path now flows retrieve -> is_relevant -> END. Irrelevant chunks are dropped, but there is still no answer generated from the surviving context.



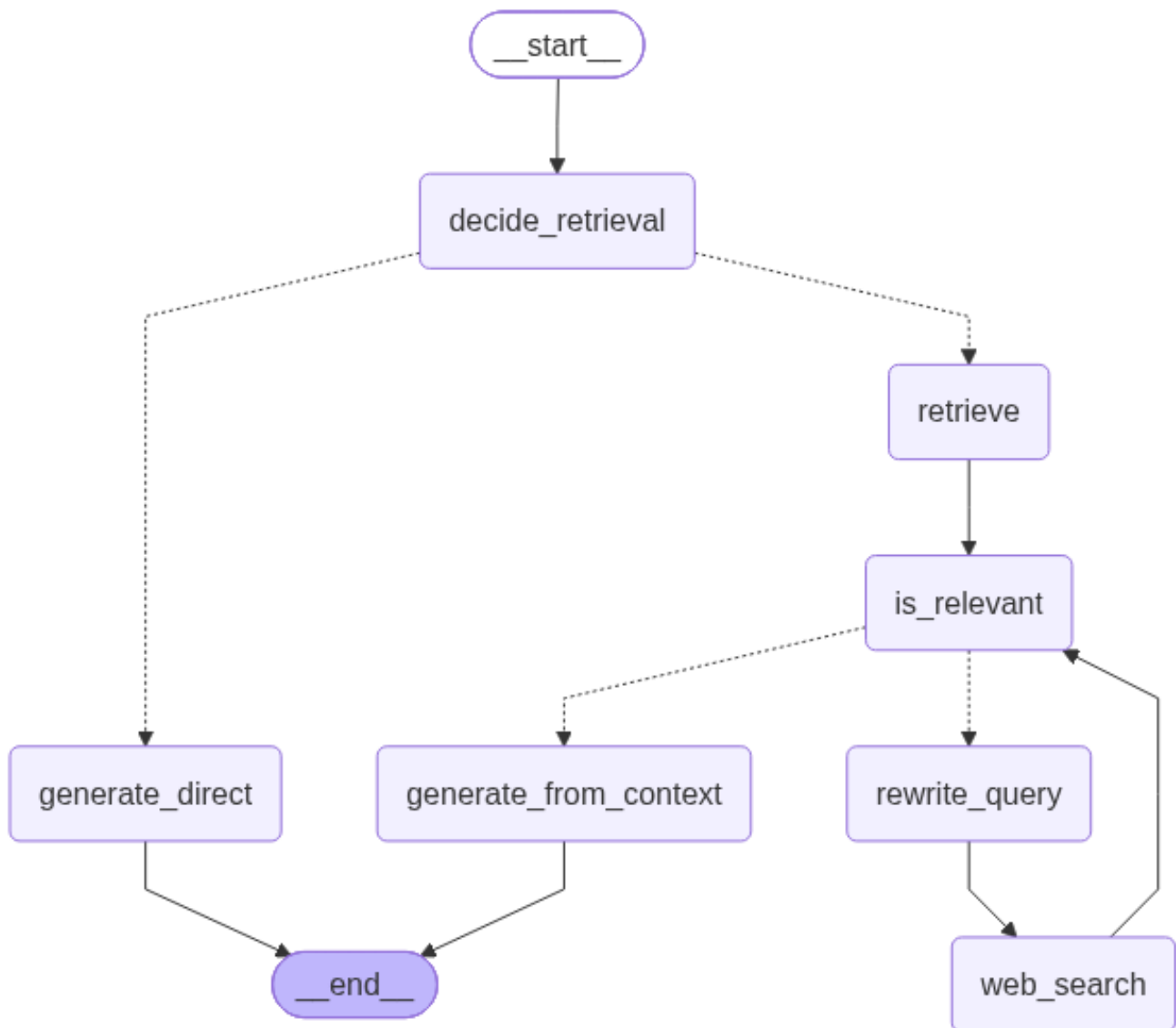
Step 3.0 - Generate From Context + Dead-End Fallback

- Adds generate_from_context: composes the final answer strictly from relevant_docs.
- Adds a no_relevant_docs fallback: if nothing relevant was found, it just returns 'No relevant document found.' and the graph ends - there is no recovery path.



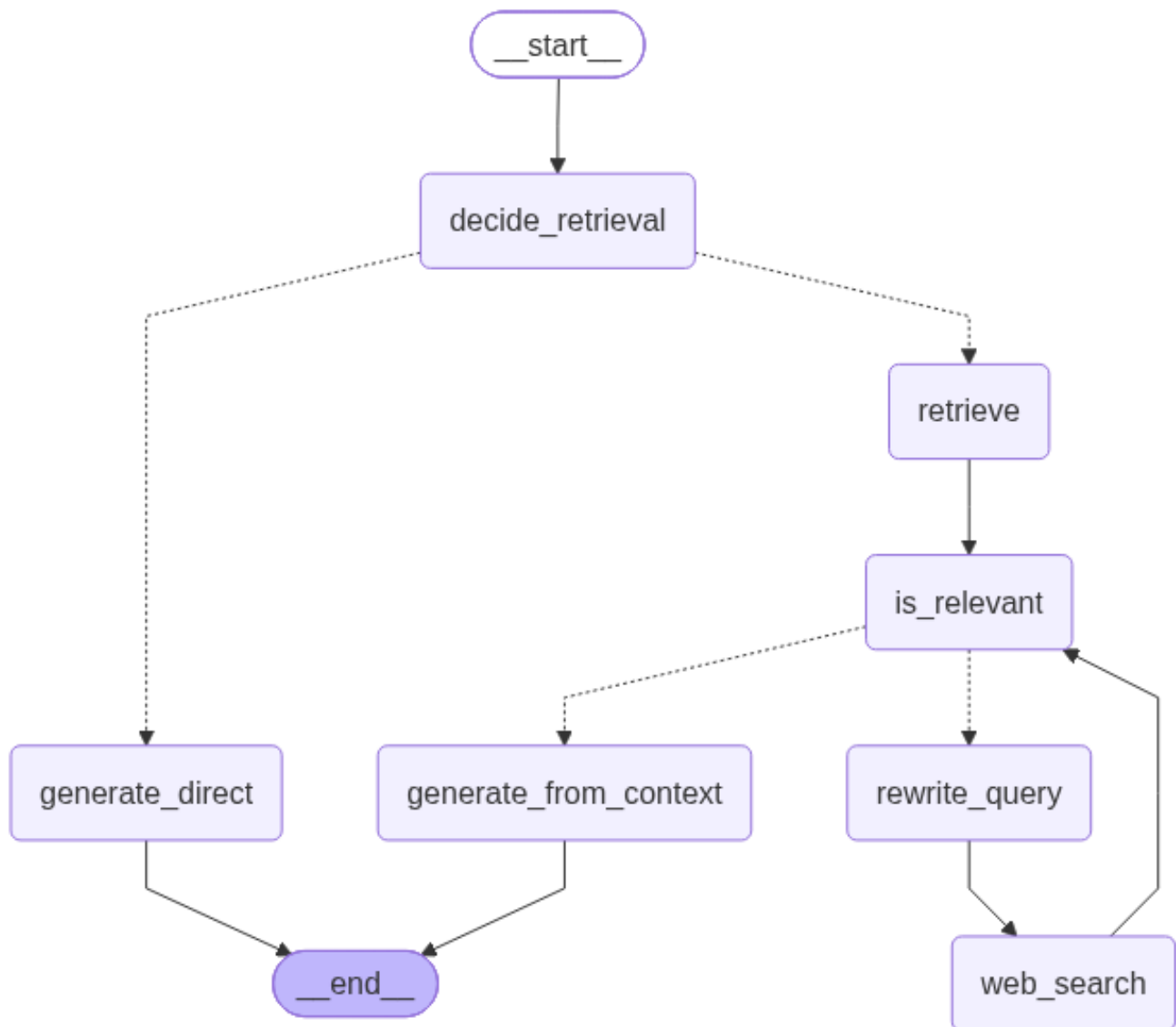
Step 3.1 - Bounded Web-Search Fallback Loop

- Replaces the dead-end fallback with a real recovery loop: `rewrite_query` -> `web_search` (Tavily) -> `is_relevant`, retried up to `MAX_WEB_RETRIES = 2`.
- As soon as relevant docs are found (local or web) it proceeds to `generate_from_context`; once retries are exhausted it still falls through to `generate_from_context`, which safely reports 'no answer' instead of dead-ending on a separate node.



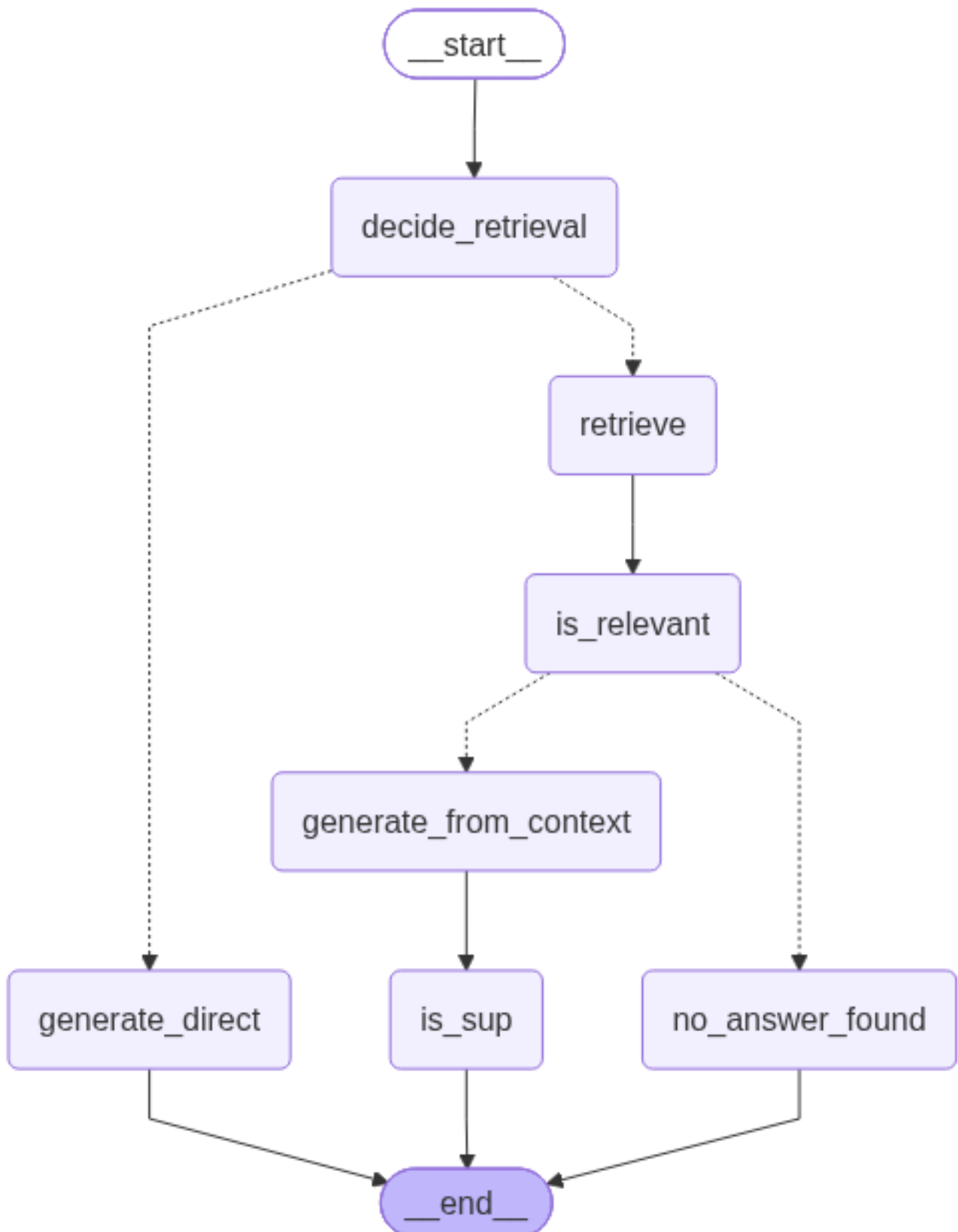
Step 3.2 (web) - Recency-Aware Web Fallback

- Same bounded rewrite -> search -> relevance loop as 3.1, tuned for genuinely open-domain, time-sensitive questions (e.g. news, live scores).
- The query-rewrite prompt appends '(last 30 days)' when the question implies recency, and the retry cap is raised to MAX_WEB_RETRIES = 5 to give the search more chances to succeed.



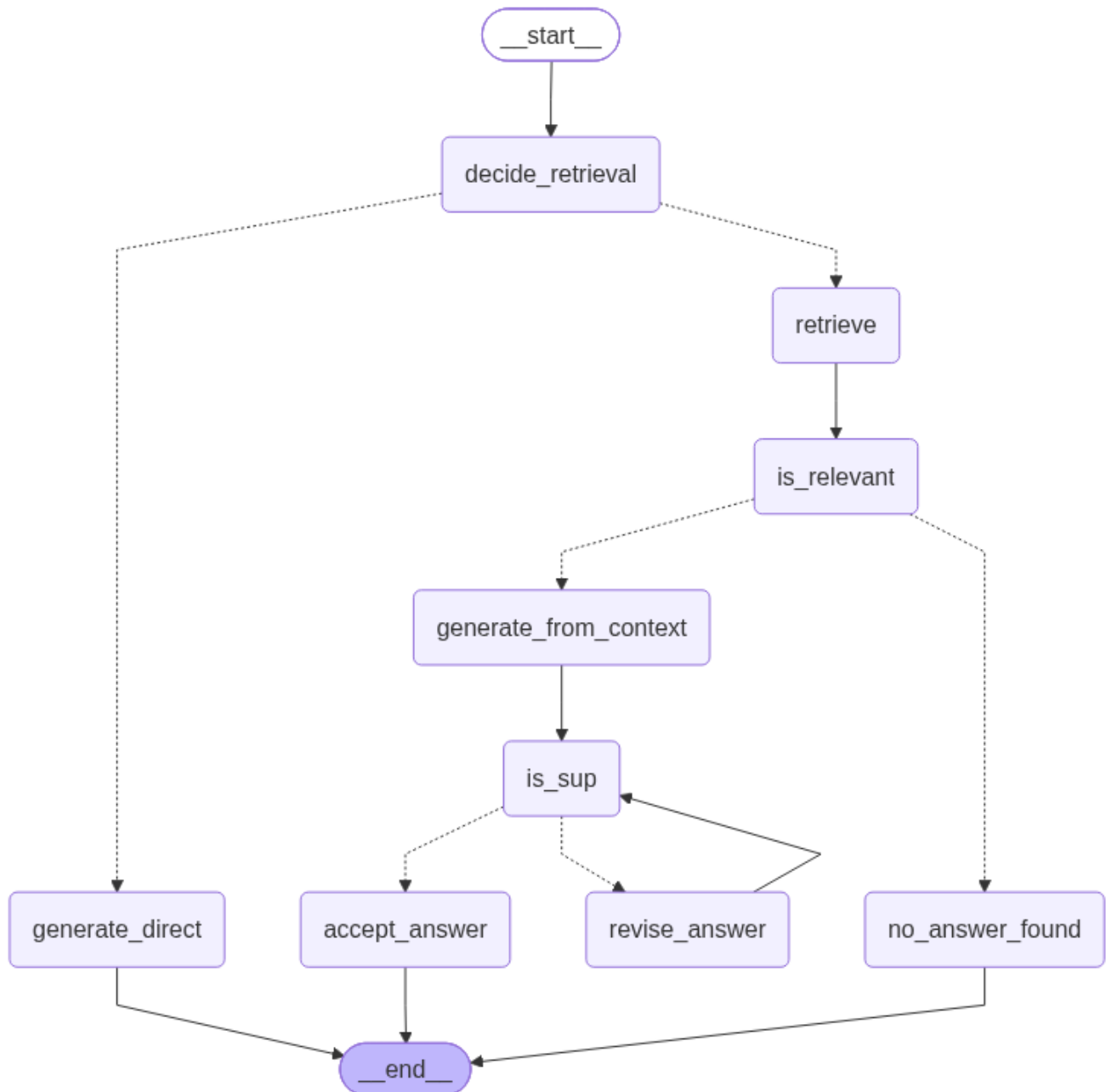
Step 4 - Post-Generation Verification (IsSUP)

- After `generate_from_context`, adds `is_sup`: an LLM checks whether the generated answer is `fully_supported`, `partially_supported`, or `no_support` against the retrieved context - catching hallucinated or unsupported claims.
- This is a single verification pass only - the result is recorded, but nothing acts on it yet (no revision loop).



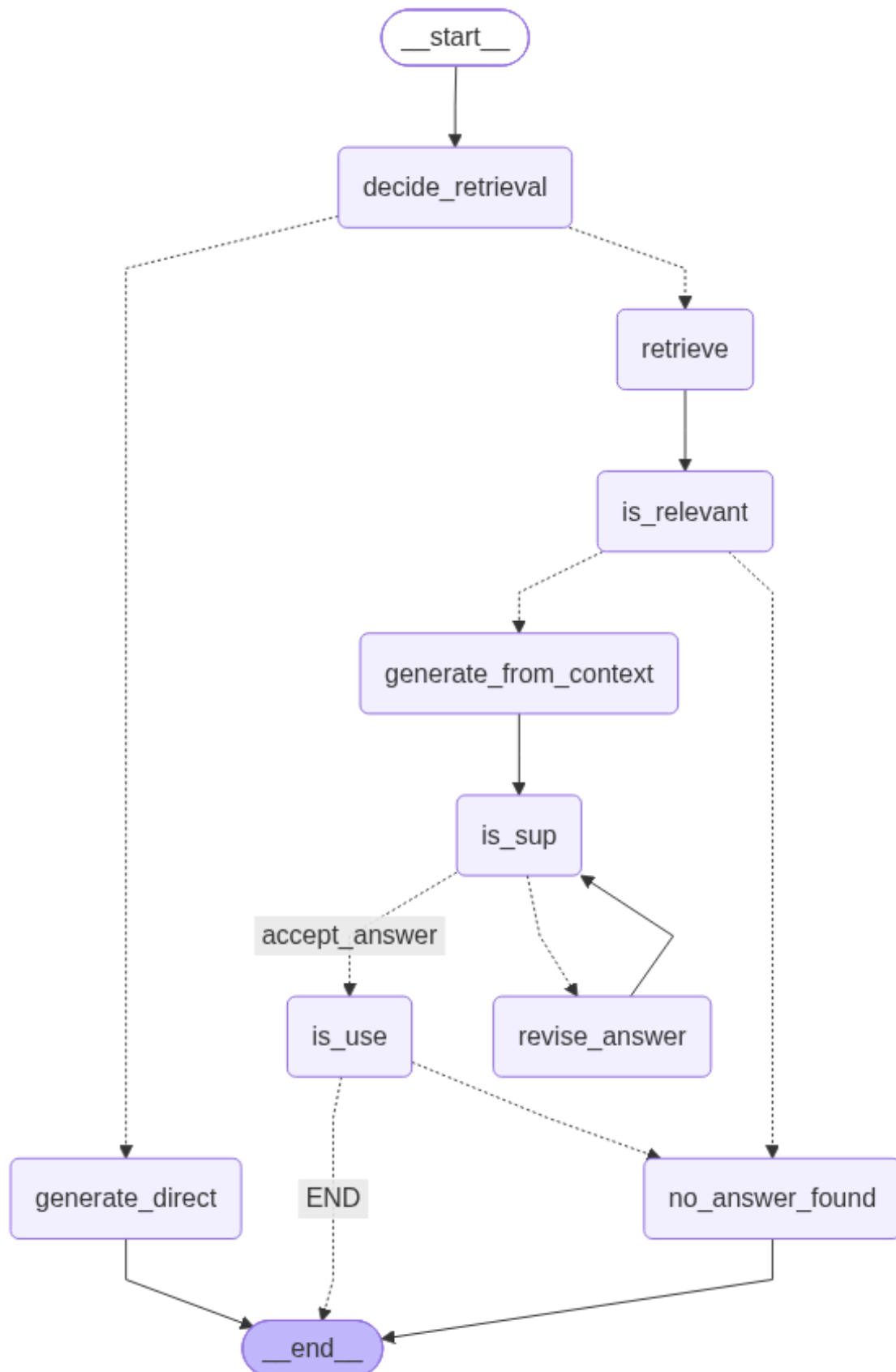
Step 5 - Verify -> Revise Loop

- Turns IsSUP into a real self-correction loop: is_sup routes to accept_answer if fully_supported, otherwise to revise_answer.
- revise_answer rewrites the answer as strict quotes taken only from the context, then loops back to is_sup - up to MAX_RETRIES = 10 - before finally being accepted.



Step 6 - Usefulness Check (IsUSE)

- Once an answer is accepted as fully supported, adds is_use: a second LLM judge asks 'does this answer actually address the question?' - a check separate from groundedness.
- If the answer is grounded but not useful (e.g. a technically correct but off-target quote dump), it routes to no_answer_found instead of returning it as-is.



Step 7 - Retrieval Query Rewriting Loop

- Closes the loop end-to-end: if `is_use` says `not_useful`, instead of giving up immediately, `rewrite_question` crafts a better `retrieval_query` (keyword-optimized for the vector store) and sends the flow back to `retrieve`.
- Bounded by `MAX_REWRITE_TRIES = 3`, this lets the system recover from a bad initial retrieval, not just a badly-worded answer - completing the full self-RAG cycle: `retrieve` -> `filter` -> `generate` -> `verify` -> `revise` -> `check usefulness` -> `re-retrieve`.

